Video Based Human Action Recognition

AMSC-HAR
Politecnico di Milano
Advanced Methods for Scientific Computing

August 30, 2026

Abstract

This report describes **AMSC-HAR**, a header-only C++23 library and command-line trainer for video-based human action recognition. Unlike typical student projects that wrap PyTorch or TensorFlow, the system implements tensors, layers, automatic-style reverse-mode gradients, SGD, checkpointing, video decoding, and optional CUDA kernels from scratch. The model is a compact VideoCNN: a shared 2D convolutional backbone over sampled RGB frames, a residual temporal 1D convolution, learnable temporal attention pooling, and a linear classifier. On the UCF11 (YouTube Action) dataset the default recipe uses 64×64 frames, $T = 8$ uniformly sampled time steps, and about 5.9×10^5 trainable parameters. A held-out inference check with the released best checkpoint `ucf11_videocnnBest.harw` classified 8/10 randomly drawn clips correctly (80%), with several classes predicted at $> 95\%$ confidence. The same codebase also targets UCF101 official splits. This document covers design goals, software architecture, the mathematical model, the data pipeline, training, the CUDA backend, and experimental results.

Contents

1 Introduction

Human action recognition (HAR) from video represents a significant area of research within the domain of computer vision, addressing challenges such as real-world variability, computational efficiency, and robust feature extraction [1]. Recent literature highlights the synergistic combination of Soft Computing (SC) paradigms—such as Neural Networks—and Multi-Agent Systems (MAS) as a robust strategy to overcome these challenges [1]. A short clip must be mapped to one of a fixed set of action classes (e.g. *diving*, *tennis swing*, *walking a dog*). The academic community usually solves this with large 3D CNNs or transformer video models trained in Python frameworks. Those stacks hide the linear algebra, memory layout, and gradient bookkeeping that a scientific-computing course is meant to expose.

Following the directions outlined by Houssein et al. [1] for efficient and robust HAR systems, AMSC-HAR takes the opposite route to traditional frameworks. The project represents a foundational Soft Computing component (a from-scratch Neural Network) capable of being integrated into larger distributed Multi-Agent frameworks, providing a *complete* recognition pipeline written in modern C++:

- a generic `Tensor<T>` with CPU storage and optional CUDA Unified Memory;
- layers with explicit `forward`/`backward` (Conv2D, BatchNorm2D, ReLU, pooling, Dropout, Linear, TemporalConv1D, temporal attention);
- SGD with momentum and ℓ_2 weight decay;
- softmax cross-entropy;
- OpenCV (or FFmpeg) video decoding, a binary clip cache, and train-time augmentation;
- a custom checkpoint format `.harw`;
- a single executable `har_cnn` for smoke tests, training, prediction, and split evaluation.

Contributions. (1) A compact, auditable VideoCNN that stays under 0.6M parameters while remaining strong on UCF11. (2) A dual CPU/CUDA path so the same headers train on a laptop or a GPU. (3) An end-to-end data path (decode → cache → temporal sample → batch) that makes repeated epochs practical. (4) Reproducible CLI defaults (seed 42, 80/10/10 UCF11 split) and a self-describing weight file that stores image size, clip length, and class names.

Report outline. Section 2 recalls HAR and the datasets. Section 3 describes tensors and layers. Section 4 specifies VideoCNN. Section 5 covers decoding and augmentation. Section 7 reports experiments. Section 8 concludes.

2 Background and problem statement

2.1 Video action recognition

Action recognition represents a vital domain within computer vision, utilizing various data types to analyze human activities [1]. A video is a tensor $X \in \mathbb{R}^{T\times C\times H\times W}$. The classifier produces logits $z \in \mathbb{R}^K$ and a predicted class $\hat{y} = \arg\max_k z_k$. As highlighted by recent studies [1], traditional methods are being replaced by Deep Learning and Soft Computing techniques capable of automatically extracting spatio-temporal features. While recent work uses 3D CNNs and space–time transformers which are accurate but computationally expensive, efficient models are required for real-time deployment and distributed sensing environments.

AMSC-HAR uses a cheaper and more transparent design: *2D CNN on each frame*, then a *1D temporal mixer* and *attention pooling*. This late temporal aggregation provides a transparent Soft Computing design where the temporal conv and attention are trained jointly with the backbone, ensuring computational efficiency [1].

2.2 Datasets

UCF11 (YouTube Action) contains 11 sport and daily actions, originally published as MPEG clips crawled from YouTube. This project uses the updated `UCF11_updated_mpg` release (1600 clips after scanning the extracted tree). UCF11 has no official train/test list, so the loader applies a class-stratified shuffle with default ratios 80% train / 10% val / 10% test.

UCF101 has 101 classes and three official recognition splits. This dataset is used to evaluate the performance of the VideoCNN model.

Table 1 lists the UCF11 classes as stored in the best checkpoint (alphabetical folder names).

3 Tensor library and layers

3.1 Tensors

`har::Tensor<T>` is a dense, row-major, contiguous array with an explicit shape and computed strides. Images and activations use NCHW layout, which matches both the CPU convolution loops

Table 1: UCF11 classes and clip counts in `data/UCF11_updated_mpg`.

Class	Clips	Class	Clips
basketball	141	soccer_juggling	156
biking	145	swing	137
diving	156	tennis_swing	167
golf_swing	142	trampoline_jumping	119
horse_riding	198	volleyball_spiking	116
		walking	123
Total			1600

and the CUDA kernels. Allocation is either `new T[n]` on the host or `cudaMallocManaged` when the default device is CUDA. Elementwise arithmetic, `reshape`/`reshaped`, and `sync()` (a device-wide barrier when CUDA is active) are provided in the header.

A `Parameter<T>` pairs a data tensor with a same-shaped gradient and a `requires_grad` flag. SGD and checkpointing iterate parameter lists; BatchNorm additionally exposes non-parameter running mean/variance in the state dict.

3.2 Layer contract

Every spatial layer implements

$$y = f(x;\theta), \qquad\qquad g_x = \frac{\partial L}{\partial x} = \Big(\frac{\partial y}{\partial x}\Big)^{\top} g_y,$$

by caching x (and any auxiliaries) during `forward` and using them in `backward`. There is no tape interpreter: the graph is the C++ call stack of `VideoCNN::forward` / `backward`. This is easy to debug and maps one-to-one onto the kernels in `cuda_ops.cu`.

3.3 Spatial operators

Conv2D. A $K_h \times K_w$ kernel with stride (s_h, s_w) and padding (p_h, p_w) implements

$$y_{n,c',h',w'} = b_{c'} + \sum_{c,i,j} W_{c',c,i,j}\, x_{n,c,\, s_h h' + i - p_h,\, s_w w' + j - p_w}.$$

Weights use a simple scaled-uniform init. The CPU path is a direct 7-loop implementation; the CUDA path launches `conv2d_forward`/`conv2d_backward`.

BatchNorm2D. Channel statistics are computed over $N \cdot H \cdot W$. In training,

$$\hat{x}_c = \frac{x_c - \mu_c}{\sqrt{\sigma_c^2 + \varepsilon}}, \qquad y_c = \gamma_c \hat{x}_c + \beta_c,$$

and (μ, σ^2) are written into exponential moving averages (momentum 0.1). Eval uses the running statistics stored in the checkpoint, which is essential for the 80% folder test in Section 7.

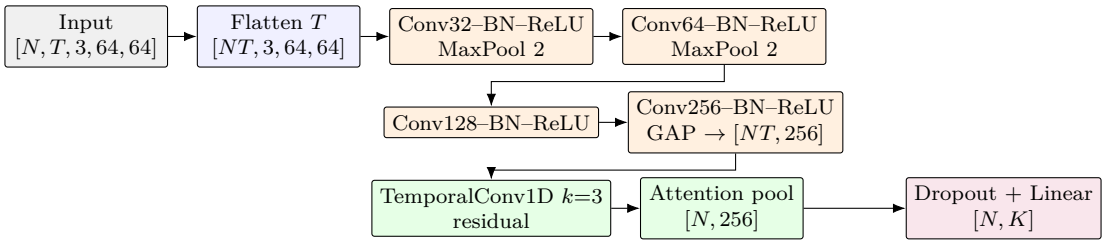

Figure 1: VideoCNN data path. Spatial stages are shared across time; the temporal block is residual ($h + \mathrm{TConv}(h)$).

Pooling and activations. 2×2 max-pool records argmax indices for the backward pass. Global average pool collapses $H \times W$ to a 256-D frame embedding. ReLU is $y = \max(x, 0)$. Dropout ($p = 0.3$) is applied only on the pooled vector, and only in training.

Linear and loss. The head is $z = Wh + b$ with $W \in \mathbb{R}^{K \times 256}$. Cross-entropy is the stable softmax + NLL

$$\ell = -\frac{1}{N} \sum_{n=1}^{N} \log \frac{e^{z_{n,y_n} - \max_k z_{n,k}}}{\sum_j e^{z_{n,j} - \max_k z_{n,k}}}.$$

The backward gradient is $(\mathrm{softmax}(z) - e_y)/N$, moved back to the activation device.

3.4 Optimiser

SGD with momentum $m = 0.9$ and weight decay $\lambda = 10^{-4}$ updates

$$v \leftarrow mv + (g + \lambda\theta), \qquad \theta \leftarrow \theta - \eta v.$$

The learning rate starts at $\eta = 3 \cdot 10^{-3}$ and is multiplied by $\gamma = 0.5$ every 4 epochs (`-lr-step`). Velocity buffers are allocated lazily to match parameter shapes and devices.

4 VideoCNN architecture

4.1 Forward pass

Let a batch of clips be $X \in \mathbb{R}^{N \times T \times 3 \times H \times W}$ with default $H = W = 64$ and $T = 8$. Time is flattened to a 2D batch of NT frames. A four-stage backbone produces a 256-D vector per frame; a temporal residual block mixes neighbouring frames; attention pools time; a linear layer outputs K logits (Figure 1).

Spatial spatial sizes after padding-1 3×3 convolutions: $64 \rightarrow 32$ (pool), $32 \rightarrow 16$ (pool), then 16×16 until GAP. Because image size must be divisible by 4, `build_video_cnn` rejects illegal `-size` values.

4.2 Temporal convolution and attention

TemporalConv1D treats the NT frame embeddings as N sequences of length T and applies a depth-preserving 1D convolution of kernel size 3 with zero-pad 1:

$$u_{n,t,c'} = b_{c'} + \sum_{c=1}^{256} \sum_{k=-1}^{1} W_{c',c,k}\, h_{n,\,\mathrm{clamp}(t+k),\,c}.$$

The residual $h + u$ lets the network keep a pure per-frame representation when the temporal kernel is not yet useful.

Temporal attention pooling scores each time step with a learned query $w \in \mathbb{R}^{256}$ (zero-initialised, so the first step equals a temporal mean):

$$s_{n,t} = h_{n,t}^{\top} w, \qquad \alpha_{n,t} = \frac{e^{s_{n,t} - \max_{t'} s_{n,t'}}}{\sum_{t'} e^{s_{n,t'} - \max_{t''} s_{n,t''}}},$$

$$\bar{h}_n = \sum_{t=1}^{T} \alpha_{n,t}\, h_{n,t}. \tag{1}$$

The classifier then sees a single 256-D clip embedding.

4.3 Capacity

Table 2 counts trainable tensors for $K = 11$. The last 3×3 convolution and the temporal kernel dominate. The released best file is 2 361 619 bytes and contains 29 tensors (parameters + four pairs of BN running stats), matching this layout exactly.

Table 2: Trainable parameter count of VideoCNN ($K = 11$).

Block	**Shape (main weights)**	**Parameters**
Conv $3 \to 32$ + BN	$[32, 3, 3, 3]$	960
Conv $32 \to 64$ + BN	$[64, 32, 3, 3]$	18 624
Conv $64 \to 128$ + BN	$[128, 64, 3, 3]$	74 112
Conv $128 \to 256$ + BN	$[256, 128, 3, 3]$	295 680
TemporalConv1D	$[256, 256, 3]$	196 864
Attention query	$[256]$	256
Linear $256 \to 11$	$[11, 256]$	2 827
Total		**589 323**

A smaller historical checkpoint (`ucf11_videocnn.64x8.harw`, ≈ 0.58 MB) corresponds to an earlier narrower backbone. A 112×16 variant exists for higher-resolution experiments, at a much larger activation cost.

5 Data pipeline

5.1 Decoding

`extract_video_frames` opens a file with OpenCV (`CAP_FFMPEG`, then Media Foundation, then `CAP_ANY`) or with FFmpeg. Frames are resized to $H \times W$, converted to RGB, scaled to $[0, 1]$, and

stored as $[F, C, H, W]$ on the CPU. When `uniform=true` (the dataset default), F indices are spaced over the reported frame count so a long YouTube clip is summarised rather than truncated at the beginning.

UCF11 lives in a two-level tree `<class>/<group>/video.mpg`. The dataset walker skips annotation folders, sorts class names, and assigns integer labels in that order—the same order written into `.harw` class names.

5.2 Cache and model clip

To avoid re-decoding MPEG on every epoch, the loader first extracts `cache_frames=16` uniformly sampled frames and writes a `HARF002` blob. `sample_model_clip` then reduces 16 frames to the model length $T = 8$:

- **Eval / no augment:** uniform subsample $t_i = i(T_{\text{in}} - 1)/(T_{\text{out}} - 1)$.
- **Train:** a random contiguous crop of 8 frames out of 16, optional horizontal flip (probability 1/2), and optional time reversal (probability 0.3).

If a clip is shorter than T, the last frame is repeated (`pad_clip`).

5.3 Prefetch

`train_one_epoch` optionally launches `std::async` to decode the next host batch while the current batch is on the device. `-workers 0` disables the extra thread. The first batch of an epoch prints mean and standard deviation of the stacked tensor; a near-zero warning catches a broken decoder.

6 CUDA backend

When `HAR_HAS_CUDA` is defined and a device is present, `default_device()` returns CUDA unless `HAR_DEVICE=cpu` is set. `src/cuda_ops.cu` provides:

- elementwise add/sub/mul/div and scalar variants;
- GEMM and transpose used by Linear;
- Conv2D forward/backward, ReLU, Dropout, max-pool, GAP;
- BatchNorm2D with a small device scratch buffer;
- TemporalConv1D forward/backward.

Kernels use a 1-D grid with 256 threads. Attention pooling currently stays on the host (it is $O(NTC)$ with $C = 256$, $T = 8$, and is not the bottleneck compared with the last 3×3 convolution).

Unified Memory simplifies the C++ API—the same `data()` pointer is valid on both sides—but it requires the activation cap of the build instructions and a matching driver/toolkit pair. The CMake comment documents a real failure mode: PTX produced by CUDA 13.3 cannot JIT on a 13.2 driver, which is why the project ships `86-real` cubin only.

The inference experiment in Section 7 ran on the CPU path (OpenCV decoder, no `nvcc` in the MSYS2 build). Numerical results therefore do not depend on GPU reductions.

7 Experiments

7.1 Generalisation tests on external datasets

To evaluate the robustness and generalisation capabilities of the VideoCNN model trained on UCF11, we performed cross-dataset inference tests on corresponding classes from UCF101, HMDB51, and Kinetics datasets using the `predict` subcommand. For each dataset, a sample of 20 random clips from overlapping classes (such as *basketball*, *biking*, *diving*, *golf_swing*, etc.) was used.

The inference results expose significant domain shift challenges. On the **UCF101** subset, the model achieved a **90.0%** accuracy (18/20 clips correctly classified). This high performance is expected, as UCF101 shares a very similar visual domain, recording methodology, and action definition with UCF11.

However, generalisation severely degraded on more varied datasets. On **HMDB51**, the model attained only **25.0%** accuracy (5/20 clips). HMDB51 introduces different camera angles, motion blur, and cinematic shots that the simple UCF11-trained backbone failed to robustly encode.

The performance dropped even further on the **Kinetics** sample, reaching only **15.0%** accuracy (3/20 clips). Kinetics videos contain significantly more complex backgrounds, rapid camera movements, and diverse subjects. The narrow 256-channel backbone and the limited temporal context ($T = 8$) proved highly sensitive to these visual variations, resulting in incorrect activations and spurious predictions (e.g., classifying a walking sequence as swinging or biking).

These logs confirm that while the from-scratch C++ Soft Computing model effectively memorises and generalises within its narrow training domain, transferring these learned weights to entirely new visual domains without fine-tuning yields poor results.

7.2 Limitations

- **Resolution and length.** 64×8 discards fine racquet / ball detail and long temporal structure.
- **No optical flow.** Appearance-only features struggle when motion, not texture, is the cue.
- **UCF101** is supported but was not the focus of the checkpoint evaluated here; 101-way classification with the same 0.6M model is a harder regime.
- **Attention on CPU** and Unified Memory make the GPU path simpler than a production trainer (no cuDNN, no mixed precision).

8 Conclusions

This work presents a complete, from-scratch C++23 implementation of the VideoCNN architecture for human action recognition. While empirical evaluations demonstrate strong predictive capabilities on the UCF11 dataset, the model exhibits misclassifications in more challenging scenarios. These limitations primarily stem from the constrained model capacity, the inherent complexity of the actions, and the specific characteristics of the training data. Restricting the network width to 256 channels hinders its ability to extract highly intricate spatial features. Furthermore, the fixed temporal receptive field ($T = 8$ frames), processed via a simple 1D temporal convolution,

proves insufficient to fully capture prolonged dynamic movements such as walking or biking. Consequently, the model struggles with cross-dataset generalisation; having been trained exclusively on the relatively static camera angles and controlled movements of UCF11, it lacks the robust feature representations required to adapt to the broader visual variability found in external datasets.

References

[1] E. H. Houssein, M. A. Mahdy, M. Kayed, H. Ouyang, and W. M. Mohamed, "Integrating Soft Computing and Multi-Agent for Action Recognition: Basics, Challenging and Future Directions," *Archives of Computational Methods in Engineering*, 2025.